CS 7180: Claude Code Workflows & Development Practices

John Alexis Guerra Gomez
jguerra at northeastern.edu
Class: **johnguerra.co/classes/aiCoding_spring_2026**
Slides: **johnguerra.co/lectures/ai_assisted_coding**

What We'll Cover Today

1. Where We Are -- Week 11 checkpoint
2. Explore -> Plan -> Implement -> Commit
3. TDD with Claude Code
4. Git & GitHub Integration
5. CI/CD as Claude Code Workflows
6. Hooks for Quality Enforcement
7. Visual Communication & Debugging
8. Non-Interactive Mode & Scripting
9. Evaluation Systems (LLM-as-Judge)
10. Hands-on Lab

Where We Are

> *Week 11 -- Professional Workflows with Claude Code*

Recap: Week 10 Foundations

Last week you learned the **what** of Claude Code:

- The agentic loop (think -> tool -> observe -> repeat)
- CLAUDE.md for persistent project context
- Context management (/clear, /compact, checkpoints)
- Permission model and sandboxing
- Thinking and plan modes

This week: The **how** -- professional workflows for real engineering.

The P3 Journey

P3 is now in Sprint 1. Your 4-person team should have:

- A GitHub repo with a CLAUDE.md
- A project board with issues
- A sprint backlog with assigned tasks

Today's workflows are what you'll use **daily** for the rest of P3.

Explore -> Plan -> Implement -> Commit

> *Anthropic's recommended 4-phase development pattern*

The Four Phases

Anthropic recommends structuring every feature as four distinct phases:

Each phase has a different **mental mode** and **Claude Code usage pattern**.

Phase 1: EXPLORE

Goal: Understand the problem and existing code before changing anything.

```
> Explore the authentication system. What middleware
is used?
> How are sessions managed? Where are the route
guards?
```

Key behaviors:

- Claude Code reads files, greps patterns, traces dependencies
- You get a mental model without writing any code
- Save findings to a file if the task is complex

```
> Write your findings to docs/auth-analysis.md
> /clear
```

Phase 2: PLAN

Goal: Design the solution before writing code.

```
> (plan) Add OAuth2 login with Google. The app
currently
> uses session-based auth with express-session.
```

Claude Code will:

1. Read relevant files (auth routes, middleware, config)
2. Propose a step-by-step implementation plan
3. Wait for your approval before writing any code

Review the plan. Redirect if the approach is wrong. This is cheaper than fixing code later.

Phase 3: IMPLEMENT / Phase 4: COMMIT

IMPLEMENT: After approving the plan, Claude Code creates/modifies files, installs dependencies, runs tests, and fixes issues along the way. Your role is to monitor and intervene if needed.

```
> Wait -- use passport-google-oauth20, not the
deprecated package
```

COMMIT: Ship a clean, atomic commit and PR.

```
> Create a commit for the Google OAuth integration
> Push this branch and create a PR with a description
```

Claude Code stages relevant files, writes a descriptive message, and creates the commit.

Why This Pattern Works

Without the pattern:

```
> Add OAuth login
(Claude Code starts coding immediately, makes
assumptions,
 creates a mess across 12 files, hard to review)
```

With the pattern:

Each phase produces a **reviewable checkpoint**. You catch problems early when they're cheap to fix. **Explore** catches wrong assumptions. **Plan** catches wrong approaches. **Implement** catches wrong code. **Commit** catches wrong scope.

The /clear Trick for Complex Tasks

For large features, use `/clear` between phases:

```
Phase 1: EXPLORE  ->  save findings to a file
         /clear

Phase 2: PLAN     ->  read findings file, write plan
to a file
         /clear

Phase 3: IMPLEMENT -> read plan file, implement step
by step
         /clear

Phase 4: COMMIT   ->  review changes, commit
```

Your findings and plans **persist in files**, not in context. This lets you tackle tasks larger than the 200K token window.

TDD with Claude Code

> *"The single highest-leverage thing" -- Anthropic*

Why TDD Is Critical with AI

Anthropic calls TDD **"the single highest-leverage thing"** you can do when coding with AI.

Why? AI-generated code needs a **verification mechanism**:

- Without tests: you read code and hope it's correct
- With tests: you run them and **know** if it's correct

Tests are the **specification**. Claude Code writes code to meet the spec.

The AI-TDD Workflow

```
1. YOU write the test (the spec)
2. Commit the failing test
3. CLAUDE CODE implements the code
4. Tests pass -> commit
5. Refactor -> tests still pass -> commit
```

Example: TDD in Practice

Step 1 (you): Write the failing test

```
describe('UserService', () => {
  test('validates email format', () => {
    expect(() => createUser({ email: 'invalid' }))
      .toThrow('Invalid email format');
  });
  test('hashes password before storing', async () =>
{
    const user = await createUser({
      email: 'test@example.com', password:
'secret123'
    });
    expect(user.password).not.toBe('secret123');
  });
});
```

Step 2: Commit the test: `git commit -m "test: user creation specs"`

Step 3 (Claude Code): `> The tests in user-service.test.js are failing. Implement createUser to make them pass. Do not modify the tests.`

Property-Based Testing with fast-check

Go beyond example-based tests. **Property-based testing** generates hundreds of random inputs:

```
import fc from 'fast-check';

test('email validation rejects all non-email
strings', () => {
  fc.assert(
    fc.property(
      fc.string().filter(s => !s.includes('@')),
      (input) => {
        expect(() => validateEmail(input)).toThrow();
      }
    )
  );
});
```

Research data: Property-based testing shows **23-37% improvement** in pass rates for AI-generated code.

Mutation Testing with Stryker

Tests pass -- but are they actually testing anything? **Mutation testing** answers this:

```
npx stryker run
```

Stryker **mutates your code** (changes > to >=, removes lines, flips conditions) and checks if tests catch the mutations.

```
Mutation score: 87%
- 13% of mutations survived (tests didn't catch them)
- These are gaps in your test suite
```

Use mutation testing to evaluate the quality of your test suite, not just coverage.

Three Risks of AI-Generated Tests

Testkube identifies three key risks when AI generates tests:

Risk	Description	Mitigation
Business logic gaps	AI tests the code, not the requirement. Tests verify implementation details instead of business rules	Write test descriptions yourself; let AI implement
Dependency drift	AI mocks dependencies based on current behavior, not contracts. Mocks become stale	Use integration tests for critical paths; update mocks in CI
Subtle logic errors	Tests pass but encode wrong assumptions (off-by-one, timezone, encoding)	Review test assertions carefully; use property-based testing

Mitigation strategy: Humans write test descriptions and assertions. AI writes the implementation and boilerplate.

Git & GitHub Integration

> *Commits, branches, and PRs through Claude Code*

Claude Code Speaks Git

Claude Code has full access to `git` and `gh` (GitHub CLI). You can:

```
> Create a branch called feat/oauth-login
> Commit the current changes with a descriptive
message
> Push and create a PR
> Check the CI status of my PR
> Address the review comments on PR #42
```

Claude Code runs the appropriate git/gh commands and handles the details.

Branch-Per-Feature Workflow

```
main
  |
  +-- feat/oauth-login        (your feature branch)
  |     |
  |     +-- commit: tests
  |     +-- commit: implementation
  |     +-- commit: docs
  |     |
  |     +-- PR #42 -> main
  |
  +-- fix/session-timeout      (another branch)
        |
        +-- PR #43 -> main
```

```
> Create a branch called feat/oauth-login from main
> ... (work on the feature) ...
> Push this branch and create a PR targeting main
```

The gh CLI & PR Workflow

Claude Code uses gh (GitHub CLI) for all GitHub operations:

```
gh pr create --title "Add OAuth login" --body "..."
gh pr checks 42        # Check CI status
gh pr view 42 --comments   # Review comments
gh pr merge 42 --squash    # Merge when ready
```

A complete feature in natural language:

```
1. > Create branch feat/user-search from main
2. > (plan) Add full-text search to the users
endpoint
3. > Implement the plan
4. > Run the tests and fix any failures
5. > Create a commit for the search feature
6. > Push and create a PR with a summary of changes
7. > Address the reviewer's comments on the PR
```

CI/CD as Claude Code Workflows

> *GitHub Actions setup and AI-powered PR review*

Setting Up GitHub Actions

Ask Claude Code to create your CI pipeline:

```
> Set up GitHub Actions for this project:
> - Run tests on every PR, run linting, build check
> - Deploy to Vercel on merge to main
```

Claude Code creates `.github/workflows/ci.yml` configured for your stack.

AI-Powered PR Review (claude-code-action)

```yaml
# .github/workflows/claude-review.yml
name: Claude PR Review
on:
  pull_request:
    types: [opened, synchronize]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: anthropics/claude-code-action@v1
        with:
          prompt: |
            Review this PR for code quality,
            missing tests, and security concerns.
```

`anthropics/claude-code-action@v1` is now GA. For security-specific scanning, use `anthropics/claude-code-security-review@v1`.

Running Claude Code in CI

Use `claude -p` in your pipeline. **Scope tools with `--allowedTools`:**

```yaml
steps:
  - uses: actions/checkout@v4
  - name: Review code
    run: |
      claude -p "Review the code for issues" \
        --allowedTools "Read,Grep,Glob" \
        --output-format json
```

Read-only tools prevent Claude from modifying files or running commands in CI.

Hooks for Quality Enforcement

> *Deterministic rules that run on every tool call*

What Are Hooks?

Hooks are **deterministic scripts** that run before or after Claude Code tool calls:

```
Claude wants to edit a file
        |
        v
  PreToolUse hook runs
  (your script decides: allow or block?)
        |
     allowed?
     /      \
   yes       no
    |         |
  Edit      Block
  happens   + message
    |
    v
  PostToolUse hook runs
  (auto-format, lint, etc.)
```

Hook Types

Hook	When It Runs	Use Cases
PreToolUse	Before a tool executes	Block writes to sensitive files, enforce naming, validate commands
PostToolUse	After a tool completes	Auto-format code, run linting, update imports
Stop	Before Claude's final response	Validate output, check for todos, ensure tests pass

Exit codes:

- `0` = continue (allow the action)
- `2` = block (prevent the action, show message)

Example: Block Writes to Sensitive Files

```
// .claude/settings.json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "command": "python .claude/hooks/protect-
files.py"
    }]
  }
}
```

```
# .claude/hooks/protect-files.py
import sys, json
data = json.loads(sys.stdin.read())
path = data.get("tool_input", {}).get("file_path",
"")
protected = [".env", "secrets.json",
"production.config.js"]
if any(path.endswith(f) for f in protected):
    print(f"BLOCKED: {path}"); sys.exit(2)
sys.exit(0)
```

Example: Auto-Format After Edits

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "command": "npx prettier --write
$CLAUDE_FILE_PATH"
      }
    ]
  }
}
```

Every time Claude Code edits or creates a file, Prettier auto-formats it. No more style debates.

Hooks vs. CLAUDE.md Rules

Aspect	Hooks	CLAUDE.md
Enforcement	Deterministic -- always runs	Advisory -- AI may ignore
Reliability	100% -- exit code 2 = blocked	~90% -- depends on context
Flexibility	Rigid -- same rule every time	Flexible -- AI applies judgment
Best for	Security, formatting, hard rules	Conventions, preferences, guidance

Rule of thumb: If you would be upset when the rule is broken, use a hook. If it's a preference, use CLAUDE.md.

Visual Communication & Debugging

> *Paste screenshots, trace errors, fix with context*

Pasting Screenshots

Claude Code accepts images. Paste a screenshot directly into the terminal:

```
> [paste screenshot of error in browser]
> Fix this error
```

Claude Code will:

1. Analyze the screenshot (error message, stack trace, UI state)
2. Trace the error to source code
3. Implement a fix
4. Verify the fix

Works with: Browser errors, terminal output, UI bugs, design mockups.

The Debug Workflow

Example:

```
> I'm getting this error when I click the login
button:
> [paste screenshot]
> The error started after the last commit.
```

Claude Code checks the recent diff, traces the error, and fixes it.

Tips: Include full stack traces, mention what triggered the error, reference when it broke ("after the last commit"), and paste console output when available. Claude Code can also use browser tools (Playwright MCP) to interact with your running app directly.

Non-Interactive Mode & Scripting

> *`claude -p` for automation and pipelines*

The -p Flag

`claude -p` runs Claude Code non-interactively -- give it a prompt, get a result:

```
# One-shot task
claude -p "Explain the architecture of this project"

# Pipe input
git diff HEAD~1 | claude -p "Summarize these changes"

# From a file
claude -p "$(cat docs/migration-plan.md) -- implement
step 3"
```

No interactive session. No approval prompts. Just input -> output.

Output Formats & Fan-Out

Output formats for scripting:

```
claude -p "List endpoints" --output-format text
# plain text
claude -p "List endpoints" --output-format json
# structured
claude -p "Refactor auth" --output-format stream-json
# real-time
```

Fan-out pattern -- run multiple instances in parallel:

```
for file in src/routes/*.ts; do
  claude -p "Review $file for security issues" \
    --output-format json --allowedTools
"Read,Grep,Glob" &
done
wait
```

Scripting Safety

```
# Restrict tools -- read-only for review tasks
claude -p "Review this code" --allowedTools
"Read,Grep,Glob"

# Set a timeout
claude -p "Generate tests" --timeout 120000

# Use a smaller model for quick checks
claude -p "Quick check" --model sonnet
```

Always scope `--allowedTools` in automated contexts. Never give full tool access to unattended runs.

Evaluation Systems (LLM-as-Judge)

> *Using LLMs to evaluate LLM output at scale*

Why Evaluate AI Output?

You can't manually review everything Claude Code generates. At scale, you need **automated evaluation**. **LLM-as-Judge:** Use one LLM to evaluate the output of another.

Research shows **85% agreement** with human judgment -- higher than human-human agreement at **81%**.

Scoring Approaches

Pointwise scoring -- rate each output independently (1-5 scale):

```
Rate this code review comment from 1-5:
Comment: "This function has O(n^2) complexity due
to the nested loop. Consider using a Set for O(n)."
```

Pairwise comparison -- compare two outputs side by side and pick the better one. More reliable but slower (2x the cost).

Known Biases in LLM Judges

Bias	Description	Impact
Position bias	Prefers the first option in comparisons	Swap order and average scores
Verbosity bias	Rates longer responses higher	Normalize for length
Self-enhancement	Rates its own model's output higher	Use a different model as judge
Wrong logic analysis	Accepts plausible-sounding but incorrect reasoning (52.8% of cases)	Use binary sub-questions

These biases are well-documented. You must mitigate them for reliable evaluations.

Mitigation Strategies & Validation

Mitigate biases with:

1. **Chain-of-thought** -- judge explains reasoning before scoring
2. **Binary sub-questions** -- "Does the code handle empty input? (yes/no)"
3. **Few-shot examples** -- 3-5 examples of good/bad output with scores
4. **Position swapping** -- swap order in comparisons, average results

Validate with human labels:

Build a set of **30-50 human-labeled examples**. Run your judge on the same set.

- **>80% agreement:** Reliable
- **60-80%:** Needs prompt refinement
- **<60%:** Do not use

Hands-on Lab

> *TDD a feature and set up CI with Claude Code*

Lab Part 1: TDD with Claude Code (20 min)

1. Pick a feature from your P3 sprint backlog
2. **Write a failing test** (you write it, not Claude)
3. Commit the failing test
4. Ask Claude Code: `> Implement the code to make tests pass. Do not modify the tests.`
5. Verify tests pass, then commit

Lab Part 2: GitHub Actions + Hooks (25 min)

GitHub Actions:

1. `> Set up GitHub Actions to run tests and linting on every PR to main`
2. Review the workflow, push to a branch, create a PR, verify CI

Hooks:

1. Create a PostToolUse hook that auto-formats after edits
2. Create a PreToolUse hook that blocks writes to `.env`
3. Test both by asking Claude Code to edit a file and try modifying `.env`

This Week's Deliverables

Due This Week

HW4: Claude Code Workflow & TDD -- assigned this week

- TDD a feature using the red-green-refactor workflow
- Set up CI/CD with GitHub Actions
- Configure at least one hook
- Submit a screencast of your workflow

Weekly Quiz 11 -- Claude Code Workflows concepts
P3 Sprint 1 -- continue your team sprint

Next Week Preview

Week 12: Claude Code Extensibility

- **Skills & custom commands** (.claude/skills/)
- **Hooks deep dive** -- PreToolUse, PostToolUse, Stop
- **MCP servers** -- connecting databases, Figma, Playwright
- **Custom sub-agents** (.claude/agents/)
- **Parallel sessions & agent teams** (writer/reviewer pattern)

HW5 (Custom Skill + MCP Integration) will be assigned.

Resources

Required Reading

Resource	URL
Claude Code in Action (Modules 4, 7)	**anthropic.skilljar.com/claude-code-in-action**
Jest Documentation	**jestjs.io/docs/getting-started**
Vitest Documentation	**vitest.dev/guide**
GitHub Actions Documentation	**docs.github.com/en/actions**
Claude Code in GitHub Actions	**code.claude.com/docs/en/github-actions**

Recommended Reading

Resource	URL
Claude Code CLI Reference	**code.claude.com/docs/en/cli-reference**
Claude Code Hooks	**code.claude.com/docs/en/hooks**
Testing Library (React)	**testing-library.com/docs**
fast-check (Property Testing)	**github.com/dubzzz/fast-check**
Stryker Mutation Testing	**stryker-mutator.io**

Speaker notes